From android reverse engineering to identifying secrets, broken access control and API vulnerabilities:

A Mobile APK Vulnerability Chain

Client	Method	Severity	Findings
Redacted	**Static analysis**	**High to critical**	**5**

Executive Summary

This write-up details a multi-stage vulnerability chain discovered through static analysis of a production Android application. The initial foothold was a hardcoded staging environment file within the APK, cascaded into the discovery of a fully exposed API specification and multiple unauthenticated endpoints.

The chain enables: mass user enumeration, financial abuse via a third-party telephony gateway, and unauthenticated access to live customer personally identifiable information (PII) including full names, phone numbers, and real-time physical branch locations.

Technical Discovery

My personal workflow for android applications after information gathering usually includes static and dynamic analysis, in this case most of the findings were identified in the static analysis phase.

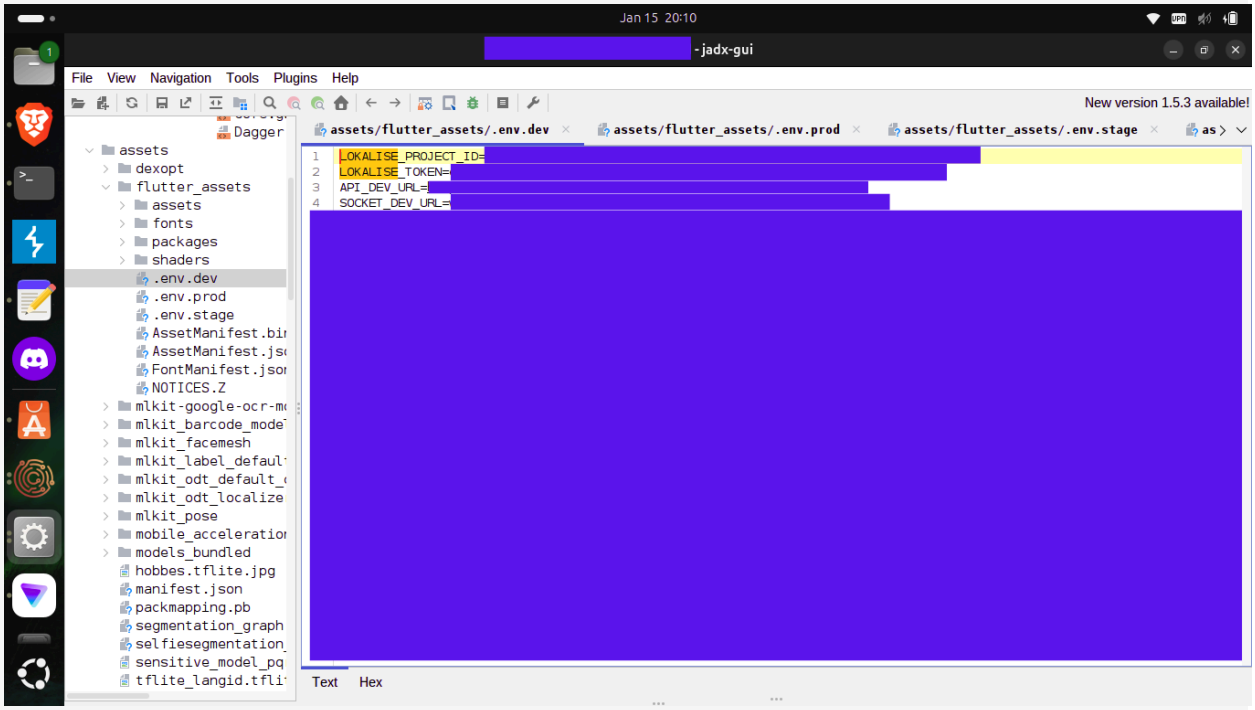
Stage 1: Decompiling the application via JADX-GUI

The production APK was decompiled using JADX-GUI. Navigating to the Flutter assets directory revealed several environment configuration files committed directly into the application bundle, including .env.stage, .env.dev, and .env.prod.

These files contained hardcoded internal base URLs and WebSocket endpoints for staging and development environments, infrastructure that was never intended to be reachable from a production client.

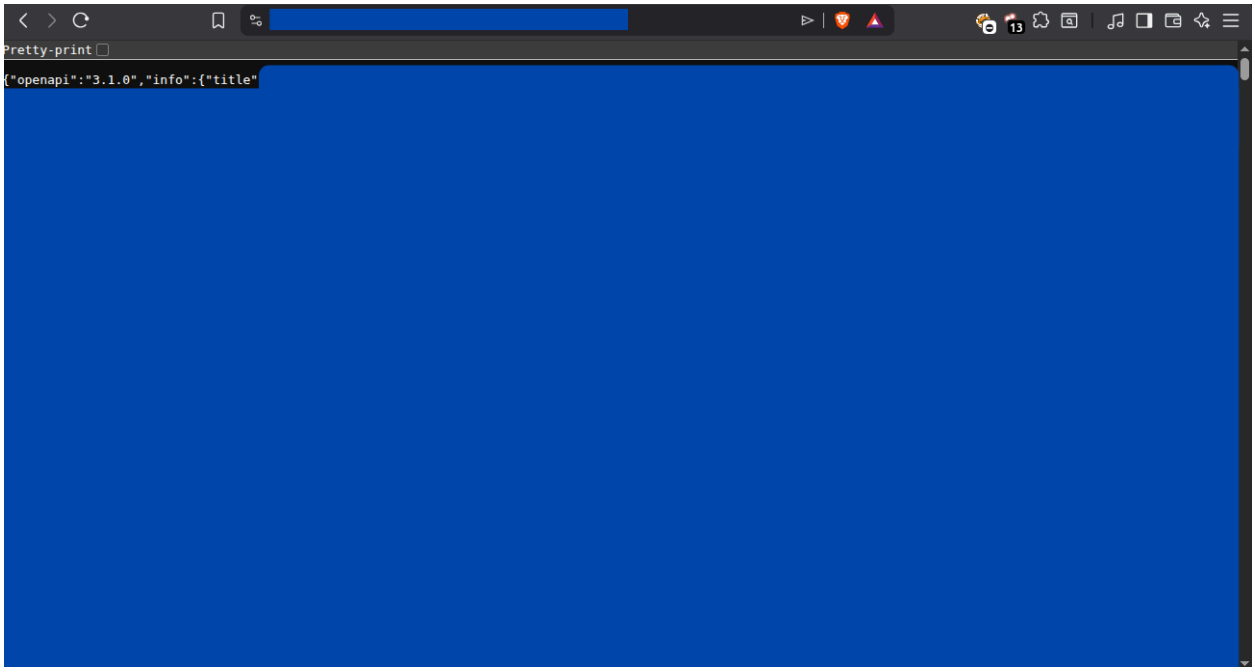
Extracted values (redacted):

```
API_STAGE_URL=https://[redacted]-dev.[target]
SOCKET_STAGE_URL=wss://[redacted]-dev.[target]
```



Stage 2 API Blueprint Disclosure

The discovered base URL exposed an internal OpenAPI/Swagger specification at /openapi.json, returning a complete JSON object documenting over 83 API endpoints. An interactive documentation interface at /docs was also publicly accessible, clearly labelling which endpoints required OAuth2 and which explicitly permitted 'Unauthorized users', and some had developers notes and internal remarks, providing a ready-made attack map for any threat actor.



Findings

F1: Hardcoded Environment Secrets in Production APK

Severity	CRITICAL
Finding	**Hardcoded environment secrets in production APK**
Endpoint	`assets/flutter_assets/ — .env.stage / .env.dev / .env.prod`
Description	Multiple environment configuration files were bundled directly into the production APK. These files disclosed internal staging and development base URLs and WebSocket endpoints, enabling any actor with the APK to identify and interact with non-production infrastructure that lacked standard security controls.

PoC	`Tool: JADX-GUI`
	`Navigation: Resources > assets/flutter_assets/`
	`Locate and open .env.stage, .env.dev, .env.prod`
	`Observe hardcoded internal URLs`

Impact Tags	*CWE-312: Cleartext Storage of Sensitive Info* | *Information Disclosure* | *Attack Surface Expansion*

F2: Unauthenticated API Documentation & Full Endpoint Map

Severity	HIGH
Finding	**Unauthenticated API documentation and full endpoint map**
Endpoint	`/openapi.json	/docs`
Description	The staging environment exposed a fully public OpenAPI specification and interactive documentation. The documentation explicitly categorised endpoints by authentication requirement, directly revealing which routes could be abused without credentials. This represents a complete attack-surface disclosure saving a real attacker significant reconnaissance time.

PoC	`curl -X GET "https://[redacted]/openapi.json"`
	
	`→ HTTP 200`
	`{83+ endpoints, auth labels, schema details}`

Impact Tags	*Information Disclosure* | *Security Misconfiguration* | *CWE-200*

F3: Unauthenticated User Enumeration

Severity	CRITICAL
Finding	**Unauthenticated user enumeration via phone oracle**
Endpoint	`POST /users/check-phone/`
Description	This endpoint accepts a phone number and returns a boolean indicating whether the number belongs to a registered user, with no authentication or rate limiting enforced. An attacker can systematically iterate through a number range to build a verified list of active customer identifiers, enabling targeted downstream attacks.

PoC	`POST /users/check-phone/`
	`Content-Type: application/json`
	
	`{"phone_number": "+XXXXXXXXXXX"}`
	
	`→ HTTP 200`
	`{"exists": false}`

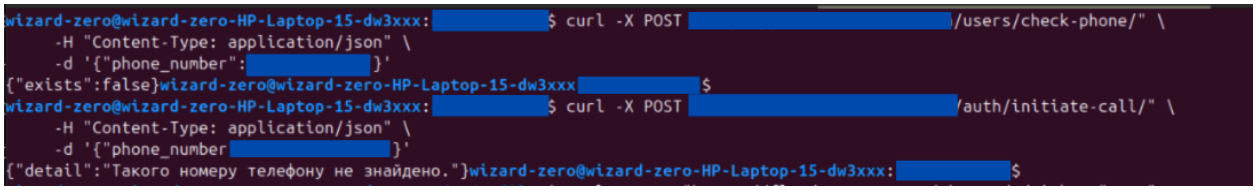
Impact Tags	*BFLA / Broken Function Level Auth* | *Mass Enumeration* | *GDPR: Data Minimisation Violation*

F4: Unauthenticated Telephony Abuse (Financial DoS)

Severity	CRITICAL
Finding	**Unauthenticated telephony abuse enabling financial DoS**
Endpoint	`POST /auth/initiate-call/`
Description	This endpoint triggers a verification call to any supplied phone number via a third-party telephony provider. No authentication is required. Since calls are billed per-request (providers such as Call2FA charge per call), an attacker can exhaust the company's telephony budget at scale and simultaneously harass arbitrary third parties with unwanted automated calls. The server response confirms telephony logic is reached before any user-existence check.

PoC	`POST /auth/initiate-call/`
	`Content-Type: application/json`
	
	`{"phone_number": "+XXXXXXXXXXX"}`
	
	`→ HTTP 200`
	`{"detail": "No such phone number found."}`
	
	`(Server reached telephony gateway before user check)`

Impact Tags	*Financial Denial of Service* | *Third-Party Harassment* | *No Auth* | *No Rate Limiting*



I used a personal number in the testing and proof of concept in order to not see any identifiable personal information, since the number was not for a registered user. The vulnerable endpoints were simply giving me access to the backend and replying, the last response in russian meant "`No such phone number found.`"

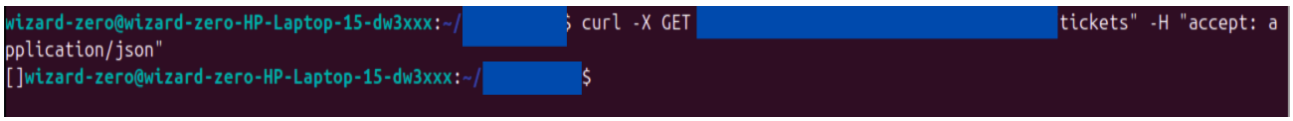
F5: BOLA + Unauthenticated PII enumeration via electronic queue:

Severity	CRITICAL
Finding	**BOLA and unauthenticated live PII leak via queue endpoints**
Endpoint	`GET /branches/{branch_id}` | `GET /queue/{branch_id}/tickets`
Description	The /branches/{branch_id} endpoint uses sequential integer IDs and returns internal branch metadata without authentication, enabling full infrastructure mapping across hundreds of physical locations. Once a valid branch ID is obtained, the queue endpoints return a live unauthenticated stream of customer data — including full names and phone numbers — for customers currently present at that branch. The API documentation explicitly labels these endpoints as accessible to 'unauthorized users', confirming this is a systemic design failure.

PoC	`GET /branches/[sequential_id]`
	`→ HTTP 200 {branch metadata}`
	
	`GET /queue/[branch_id]/tickets`
	`→ HTTP 200 [{name, phone, ...}, ...]`

Impact Tags	*BOLA / IDOR | Live PII Exposure | Real-Time Location Tracking | GDPR: Integrity & Confidentiality Violation*

As a proof of concept, I used an empty ticket which returns a 200 response with no PII.



Attack Chain & Downstream Risk

The five findings above do not exist in isolation. Together they form a complete end-to-end attack path requiring no credentials at any stage:

- Recon: Attacker decompiles the production APK and extracts .env.* files, identifying internal API base URLs and WebSocket endpoints.
- Mapping: Attacker fetches /openapi.json to obtain a complete picture of 83+ endpoints and their authentication requirements.
- Enumeration: Attacker iterates phone numbers via /users/check-phone/ to build a verified database of active customer identifiers.
- Infrastructure Mapping: Attacker iterates /branches/{id} to map every physical location and harvest branch IDs for queue access.
- Live PII Harvest: Attacker polls queue endpoints for active branch IDs, receiving real-time names and phone numbers of on-site customers.
- Exploitation: Combined data enables targeted vishing and phishing using real names and branch context; SIM-swap attacks; account takeover via intercepted 2FA, and deliberate financial exhaustion of the telephony budget.

Financial impact:

Each unauthenticated call request incurs a per-call fee from the telephony provider. Automated abuse at scale results in measurable direct financial loss with no upper bound enforced server-side.

Privacy / Regulatory impact:

Systematic exposure of customer full names, phone numbers, and real-time physical location is a direct violation of GDPR principles of data minimisation, purpose limitation, and integrity & confidentiality. This failure would likely trigger mandatory regulatory notification obligations.

Reputational impact:

Exposure of this chain constitutes a significant breach of customer trust. The architecture-level nature of several findings suggests this is not an isolated oversight but a systemic security deficit.

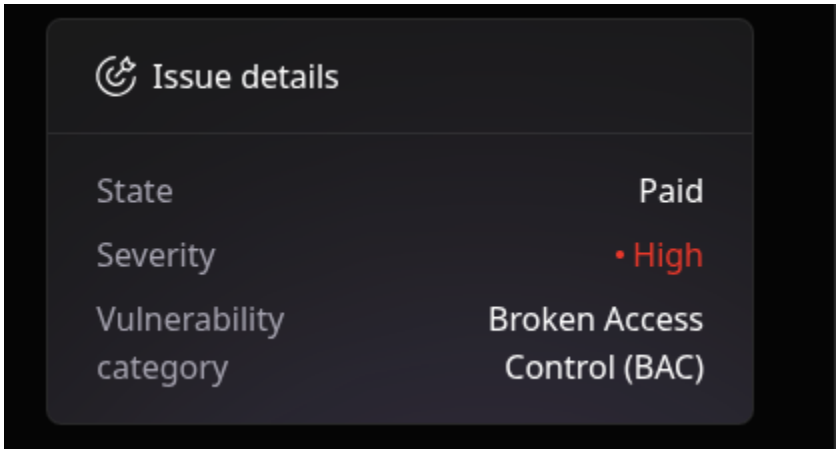
Recommended remediation:

- Remove all .env.* files from the application bundle immediately. Secrets and environment configuration must never be shipped inside a client-side artifact. Use runtime configuration injection or a secure remote config service.
- Disable or access-gate the OpenAPI/Swagger documentation on all non-local environments. API schemas should never be publicly accessible in staging or production.

- Enforce authentication (at minimum a valid bearer token) on all endpoints that interact with user data, including /users/check-phone/. Replace boolean oracle responses with opaque, non-differentiating error codes for unauthenticated requests.
- Apply strict rate limiting and authentication to /auth/initiate-call/. Telephony-triggering endpoints must require a verified session and be subject to per-user and per-IP call limits.
- Move electronic queue and branch endpoints behind authentication. If public read access is a product requirement, scope returns data to the minimum necessary, removing PII such as full names and phone numbers entirely from unauthenticated responses.
- Replace sequential integer IDs on branch and queue endpoints with non-guessable UUIDs to eliminate trivial BOLA/IDOR exploitation.

Note:

The vulnerabilities were responsibly submitted in one report considering the similarities and finally patched, the security team has only classified this report as high severity and not as critical, But on the positive side it was a great experience to work on the project and contribute to the safety of the users.



Contact information:

Linkedin: https://www.linkedin.com/in/mohamed-hadjaz-b2aba81b5/